



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Alumno:

Lara Hernandez Angel Husiel - 320060829

Arquitectura Cliente-Servidor

Grupo: 02 - Semestre: 2027-1

Tarea 3:

execve, vp y procesos padre e hijo.

Profesor:

MTRO. CARLOS ALBERTO ROMAN ZAMITIZ

Fecha de Entrega:

17 de Septiembre del 2026

Índice

1	Actividad 1:	1
2	Actividad 2	6



1. Actividad 1:

a) comenta detalladamente ambos programas

1. Programa 20_myecho.c comentado:

Listing 1: Programa 20_myecho.c comentado

```
1  /*
2  * Muestra cómo en la terminal se pasan los argumentos
3  * hacia la función main del proceso.
4  */
5
6  #include <stdio.h>
7  #include <stdlib.h>
8
9  // main recibe el número de argumentos (argc) y el vector de
   // cadenas (argv)
10 int main(int argc, char *argv[])
11 {
12     int j; // Variable iteradora para recorrer el arreglo de
           // argumentos
13
14     // Itera desde el índice 0 hasta argc - 1
15     for( j = 0; j < argc; j++ )
16     {
17         // Imprime el índice actual y la cadena correspondiente al
           // argumento
18         printf("argv[%d]: %s\n", j, argv[j]);
19     }
20
21     // Termina el proceso exitosamente devolviendo un código 0
           // al SO
22     exit( EXIT_SUCCESS );
23 }
```

2. Programa21_execve.c comentado:

Listing 2: Programa 21_execve.c comentado

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int main(int argc, char *argv[])
6  {
7      //Al iniciar se declara un arreglo de cadenas de caracteres (
           // punteros a char) llamado newargv, que contiene cinco
           // elementos: NULL, "hola", "mundo", "buen", "dia" y NULL,
           // donde al ser el primer elemento NULL, se indica que no hay
           // un nombre de programa especificado para ejecutar. Los
```



```

    elementos "hola", "mundo", "buen" y "dia" son los
    argumentos que se pasarán al programa que se ejecutará.
8 char *newargv[] = { NULL, "hola", "mundo", "buen", "dia",
    NULL };
9
10 //la sentencia de la condicional es por que se espera que el
    programa se ejecute con un argumento adicional, que es el
    nombre del archivo a ejecutar. Si no se proporciona
    exactamente un argumento (es decir, si argc no es igual a
    2), se imprime un mensaje de error en la salida estándar de
    error (stderr) y sale del programa automaticamente.
11 if( argc != 2 )
12 {
13     //Aqui es donde se imprime el mensaje de error en la salida
        estándar de error (stderr) indicando la sintaxis correcta
        para ejecutar el programa. Se utiliza fprintf para
        formatear el mensaje, donde argv[0] representa el nombre
        del programa que se está ejecutando. Luego, se llama a
        exit con EXIT_FAILURE para indicar que el programa termin
        ó con un error.
14     fprintf( stderr, "La sintaxis es: %s <archivo-a-ejecutar>\n"
        , argv[0] );
15     //Finalmente, se llama a exit con EXIT_FAILURE para indicar
        que el programa terminó con un error. Esto asegura que el
        programa no continúe ejecutándose si no se proporciona
        el argumento esperado.
16     exit( EXIT_FAILURE );
17 }
18
19 // ESTA SENTENCIA ES MUY IMPORTANTE!
20 // QUE HACE ESTA SENTENCIA?
21 // La sentencia newargv[0] = argv[1]; asigna el valor del
    segundo argumento pasado al programa (argv[1]) al primer
    elemento del arreglo newargv. Esto significa que el nombre
    del archivo a ejecutar se establece como el primer
    argumento para la función execve, que se utilizará para
    ejecutar el programa especificado por argv[1].
22 //Si lo queremos ver en terminal seria: ./programa# hola mundo
    buen dia. asi se veria en terminal.
23 newargv[0] = argv[1];
24
25 // La función execve se utiliza para reemplazar el proceso
    actual con un nuevo programa. En este caso, se pasa el
    nombre del archivo a ejecutar (argv[1]), el arreglo de
    argumentos (newargv) y NULL para indicar que no se pasan
    variables de entorno adicionales. Si la llamada a execve
    tiene éxito, el proceso actual se reemplaza por el nuevo
    programa y no se ejecuta ninguna instrucción posterior a
    esta llamada.
26 execve( argv[1], newargv, NULL );
```



```
27
28 // Aunque esta sentencia le falta un if para que no se ejecute,
29 // actualmente se puede ejecutar, por que
30 // esta fuera de la jurisdicción de execve, por lo que si
31 // execve falla, se ejecutará esta sentencia.
32 // lo que significan estas 3 líneas es: si la llamada a execve
33 // falla, se imprimirá un mensaje de error y se saldrá del
34 // programa.
35 printf("Esta sentencia no se ejecuta\n");
36 // perror imprime un mensaje de error en la salida estándar de
37 // error (stderr) que describe el error ocurrido durante la
38 // llamada a execve. Esto ayuda a identificar la causa del
39 // fallo.
40 perror("execve");
41 exit( EXIT_FAILURE );
42 }
```

- b) comprueba que el programa 20 (myecho) se puede ejecutar desde línea de comandos, pasando los argumentos que gustes.

```
● 🍏 ~/Doc/N/Cliente-Servidor/Tarea3/src 🐧 master !2 77 > ./programa20 hola profe este es el programa20
argv[0]: ./programa20
argv[1]: hola
argv[2]: profe
argv[3]: este
argv[4]: es
argv[5]: el
argv[6]: programa20
● 🍏 ~/Doc/N/Cliente-Servidor/Tarea3/src 🐧 master !2 77 > █
```

Figura 1: Ejecución del programa 20_myecho.c desde la línea de comandos con argumentos.

- c) En una segunda ejecución, pasa tu apellidopaterno apellidomaterno nombre(s). Por ejemplo:

```
● 🍏 ~/Doc/N/Cliente-Servidor/Tarea3/src 🐧 master !2 77 > ./programa20 Lara Hernandez Angel Husiel
argv[0]: ./programa20
argv[1]: Lara
argv[2]: Hernandez
argv[3]: Angel
argv[4]: Husiel
● 🍏 ~/Doc/N/Cliente-Servidor/Tarea3/src 🐧 master !2 77 > █
```

Figura 2: Ejecución del programa 20_myecho.c desde la línea de comandos con argumentos de nombre y apellidos.



- d) Al comentar el programa 21, modifica la llamada a `exec` para que quede en un bloque `if`, como lo vimos en los programas de clase, con una sentencia `exit(EXIT_FAILURE)`:

Listing 3: Programa 21_execve.c comentado

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(int argc, char *argv[])
6 {
7     //Al iniciar se declara un arreglo de cadenas de caracteres (
8     //punteros a char) llamado newargv, que contiene cinco elementos:
9     //NULL, "hola", "mundo", "buen", "dia" y NULL, donde al ser el
10    //primer elemento NULL, se indica que no hay un nombre de programa
11    //especificado para ejecutar. Los elementos "hola", "mundo", "buen"
12    //y "dia" son los argumentos que se pasarán al programa que se
13    //ejecutará.
14    char *newargv[] = { NULL, "hola", "mundo", "buen", "dia", NULL };
15
16    //la sentencia de la condicional es por que se espera que el
17    //programa se ejecute con un argumento adicional, que es el nombre
18    //del archivo a ejecutar. Si no se proporciona exactamente un
19    //argumento (es decir, si argc no es igual a 2), se imprime un
20    //mensaje de error en la salida estándar de error (stderr) y sale
21    //del programa automáticamente.
22    if( argc != 2 )
23    {
24        //Aquí es donde se imprime el mensaje de error en la salida está
25        //ndar de error (stderr) indicando la sintaxis correcta para
26        //ejecutar el programa. Se utiliza fprintf para formatear el
27        //mensaje, donde argv[0] representa el nombre del programa que se
28        //está ejecutando. Luego, se llama a exit con EXIT_FAILURE para
29        //indicar que el programa terminó con un error.
30        fprintf( stderr, "La sintaxis es: %s <archivo-a-ejecutar>\n", argv
31        [0] );
32        //Finalmente, se llama a exit con EXIT_FAILURE para indicar que el
33        //programa terminó con un error. Esto asegura que el programa no
34        //continúe ejecutándose si no se proporciona el argumento
35        //esperado.
36        exit( EXIT_FAILURE );
37    }
38
39    /*
40     * Algo que no mencione anteriormente, fue que,
41     * argv[0] almacena el nombre del ejecutable.
42     * La sentencia newargv[0] = argv[1]; asigna el valor del segundo
43     * argumento pasado al programa (argv[1]) al primer elemento del
44     * arreglo newargv. Esto significa que el nombre del archivo a
45     * ejecutar se establece como el primer argumento para la función
```



```
    execve, que se utilizará para ejecutar el programa especificado
    por argv[1].
23  */
24  newargv[0] = argv[1];
25
26  /*
27   * LLAMADA A execve DENTRO DE UN BLOQUE IF:
28   * Si execve tiene éxito, la imagen de memoria del proceso se
    sustituye
29   * y esta función NUNCA retorna.
30   * Si execve falla, retorna -1, por lo que entra al if, imprime el
    error
31   * del sistema y concluye con exit(EXIT_FAILURE).
32  */
33  if ( execve( argv[1], newargv, NULL ) == -1 )
34  {
35      perror("Error en llamada al sistema execve");
36      exit( EXIT_FAILURE );
37  }
38
39  /*
40   * Gracias al bloque if con exit(EXIT_FAILURE), este punto es
41   * completamente inalcanzable, tanto si execve tiene éxito como si
    falla.
42  */
43  printf("Esta sentencia no se ejecuta\n");
44  return 0;
45 }
```

e) Ejecuta el programa 20 llamándolo desde el programa 21

```
Problemas 4 Salida Consola De Depuración Terminal Puertos Azure Resultados De La Consulta His
~ / Doc / N / Cliente-Servidor / Tarea3 / src master !2 77 > ./programa21 ./programa20
argv[0]: ./programa20
argv[1]: hola
argv[2]: mundo
argv[3]: buen
argv[4]: dia
~ / Doc / N / Cliente-Servidor / Tarea3 / src master !2 77 > |
```

Figura 3: Ejecución del programa 20_myecho.c desde el programa 21_execve.c.

Lo que vemos aquí es que directamente la sentencia que se tenía en el argumento de newargv, se le pasó a la consola de execve, lo que hizo que el programa 20 fuera el que imprimiera toda esa sentencia de hola mundo buen día, y no el programa 21, por que execve reemplaza la imagen de memoria del proceso actual con la del nuevo programa, por lo que el flujo de ejecución del programa 21 se detiene y el control se transfiere al programa 20, que luego imprime los argumentos proporcionados.



2. Actividad 2

Toma el programa 23 y coméntalo detalladamente:

Listing 4: Programa 23_fork-exec.c comentado

```
1
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <sys/types.h>
5 #include <unistd.h>
6
7 /*
8
9  * El proposito de esta funcion es que es duplicar el proceso llamador
10  * mediante fork() y ejecuta un nuevo
11  * y lo que hara el proceso hijo es usar execvp().
12  * lo que se le pasa a la funcion es:
13  * - program: Nombre o ruta del programa a ejecutar (ej. "ls").
14  * - arg_list: Lista de argumentos terminada en NULL.
15  * y al final lo que nos dara es el PID del proceso hijo al proceso padre
16  * .:
17  */
18
19 int spawn (char* program, char** arg_list)
20 {
21     pid_t child_pid;
22
23     // Clona el proceso actual creando un proceso hijo
24     child_pid = fork ();
25
26     if (child_pid != 0)
27     {
28         // Bloque ejecutado por el proceso PADRE (child_pid contiene el PID
29         // del hijo)
30         return child_pid;
31     }
32     else
33     {
34         // Bloque ejecutado por el proceso HIJO esto es cuando el proceso hijo
35         // es creado osea un child_pid == 0
36
37         /*
38          * Si execvp() tiene éxito, la imagen del hijo es reemplazada por el
39          * programa
40          * y jamás llega a la línea siguiente.
41          * Si falla, retorna -1, se procesa el error y finaliza con exit(
42          * EXIT_FAILURE).
43          */
44         if ( execvp (program, arg_list) == -1 )
45         {
46             perror ("Ocurrió un error al ejecutar execvp");
47         }
48     }
49 }
```



```
40     exit( EXIT_FAILURE ); // Termina el proceso hijo con estado de error
41 }
42
43 // este return se pone, por que si execvp falla, el proceso hijo no
// termina y continua ejecutando el código, por lo que se pone un
// return 1 para indicar que hubo un error en la ejecución del
// programa.
44 return 1;
45 }
46 }
47
48 int main ()
49 {
50     // Lista de argumentos para el comando "ls -l /"
51     // arg_list[0] es el nombre del comando
52     // El último elemento debe ser NULL como final de la lista
53     char* arg_list[] = {
54         "ls",
55         "-l",
56         "/",
57         NULL
58     };
59
60     // El padre crea al hijo y le indica que ejecute "ls" con sus argumentos
61     spawn ("ls", arg_list);
62
63     // El proceso padre continúa inmediatamente de forma concurrente
64     printf ("Termina el proceso padre\n");
65
66     return 0;
67 }
```

Compila y ejecuta. Agrega la captura de pantalla de la ejecución.

```
● 🍏 ~ /Doc/N/Cliente-Servidor/Tarea3/src 🍏 P master !2 ?7 > gcc programa23_fork-exec.c -o programa23
● 🍏 ~ /Doc/N/Cliente-Servidor/Tarea3/src 🍏 P master !2 ?7 > ./programa23
Termina el proceso padre
total 10
drwxrwxr-x 46 root admin 1472 Sep 13 10:52 Applications
drwxr-xr-x 68 root wheel 2176 Aug 18 19:58 Library
drwxr-xr-x@ 10 root wheel 320 Aug 12 20:51 System
drwxr-xr-x 5 root admin 160 Aug 27 21:28 Users
drwxr-xr-x 4 root wheel 128 Sep 13 12:15 Volumes
drwxr-xr-x@ 39 root wheel 1248 Aug 12 20:51 bin
drwxrwxr-t 2 root admin 64 Apr 30 12:33 cores
dr-xr-xr-x 4 root wheel 4941 Sep 13 09:30 dev
lrwxr-xr-x@ 1 root wheel 11 Aug 12 20:51 etc -> private/etc
lrwxr-xr-x 1 root wheel 25 Sep 13 10:07 home -> /System/Volumes/Data/home
drwxr-xr-x 4 root wheel 128 Aug 22 12:10 opt
drwxr-xr-x 6 root wheel 192 Sep 13 10:07 private
drwxr-xr-x@ 76 root wheel 2432 Aug 12 20:51/sbin
lrwxr-xr-x@ 1 root wheel 11 Aug 12 20:51 tmp -> private/tmp
drwxr-xr-x@ 11 root wheel 352 Aug 12 20:51 usr
lrwxr-xr-x@ 1 root wheel 11 Aug 12 20:51 var -> private/var
● 🍏 ~ /Doc/N/Cliente-Servidor/Tarea3/src 🍏 P master !2 ?7 > |
```

Figura 4: Ejecución del programa 23_fork-exec.c.



Ahora bien, ya que el proceso padre no invoca la llamada `wait()` o `waitpid()`, el proceso padre y el proceso hijo compiten por la salida en la terminal, esto lo que provocara es que la línea Termina el proceso padre se imprima antes, en medio o al final de la salida del comando `ls -l /`.